DreamCycle Technical Thesis and Research Report

Author: Kenny Jin

Project: DreamCycle 0.2.1 alpha

Date: July 16, 2026

Repository: https://github.com/kenjix217/dreamcycle

License: Apache-2.0

Abstract

Local and self-hosted language-model systems are increasingly practical, but most of them still behave as stateless endpoints. They may answer well inside one conversation, yet useful corrections, successful workflows, user preferences, and domain-specific operating patterns disappear unless the host application builds a memory and learning layer around them. At the same time, naive fine-tuning can make a model worse, leak private data, or turn unreviewed conversations into training material without consent.

DreamCycle proposes a guarded learning cycle for local and OpenAI-compatible AI systems. The system stores scoped L2 episodic memory in PostgreSQL with pgvector, promotes durable L3 knowledge with provenance, retrieves bounded memory into prompts, and turns explicitly reviewed interactions into train/evaluation datasets for local adapter training. A candidate adapter is evaluated before activation, promotion is atomic, and rollback is preserved. Cloud models remain an integration boundary: DreamCycle can improve prompt context immediately and can hand approved datasets to provider-owned fine-tuning workflows, but it does not silently mutate hosted model weights.

This report frames DreamCycle as a memory-native, review-gated, evaluation-first architecture for compounding local intelligence. It summarizes the research motivation, implementation method, safety model, current verification evidence, limitations, and a roadmap for turning the alpha package into a credible open-source vendor SDK.

1. Introduction

The simplest deployment pattern for a local model is also the weakest: send a prompt to a model server and forget the interaction when the request finishes. That pattern is attractive because it preserves a clean boundary around inference, but it leaves most accumulated product experience outside the model.

The operator sees repeated mistakes, corrections, preferences, successful procedures, failed attempts, and domain-specific wording accumulate in logs. The model only sees the next prompt.

The common response is to add retrieval or to "fine-tune it." Both moves are useful, but neither is sufficient by itself. Retrieval can expose old information to the model, yet unfiltered memory can carry prompt injection, stale advice, or cross-user leakage. Fine-tuning can improve a local model, yet training on unreviewed production traces can encode bad behavior, private content, or accidental artifacts. A learning layer needs memory, review, evaluation, promotion, rollback, and identity boundaries as one system.

DreamCycle's thesis is that local AI improves more safely when model behavior is changed through a staged cycle:

Record -> Recall -> Review -> Train -> Evaluate -> Promote -> Roll back

The cycle separates three kinds of improvement. First, prompt lift changes the context sent to a model by injecting relevant memory. Second, local weight lift trains a candidate adapter from reviewed examples. Third, cloud dataset lift prepares governed examples for official provider fine-tuning workflows without turning DreamCycle into a hosted fine-tuning service.

This position is deliberately narrower than a general AI platform. DreamCycle does not own a user interface, local model server, provider account, cloud fine-tuning job, consent policy, or production authentication perimeter. It is a Python package and optional sidecar that vendors can add beside an existing system.

Contributions

- A practical L2/L3 memory model implemented directly on PostgreSQL and pgvector.
- A prompt-time recall path that can improve local or OpenAI-compatible endpoints without changing model weights.
- A review-gated dataset builder that avoids automatic training on observed conversations.
- A guarded local adapter cycle with dataset creation, training, evaluation, promotion, and rollback.
- A vendor SDK and sidecar contract that does not require JintellarCore or any other host architecture.
- A safety boundary that treats cloud fine-tuning as an explicit dataset handoff, not an automatic provider mutation.

2. Related Work and Positioning

Retrieval-augmented generation established the value of external non-parametric memory for language generation. RAG-style systems condition generation on retrieved evidence so a model does not rely only on parametric knowledge [1]. DreamCycle follows the same broad principle but applies it to user and product memory rather than a static document corpus.

Vector search systems make that memory practical. pgvector brings exact and approximate nearest-neighbor search into PostgreSQL, including cosine, L2, inner product, HNSW, and IVFFlat support [4]. HNSW itself is a graph-based approximate nearest-neighbor method designed for efficient

high-dimensional search [3]. DreamCycle uses PostgreSQL as the durable substrate because memory rows, review state, provenance, deletion, identity scope, and vectors belong in one transactional system rather than in a detached index alone.

Parameter-efficient fine-tuning makes local adaptation feasible. LoRA freezes base-model weights and trains low-rank matrices, reducing trainable parameters and memory pressure compared with full fine-tuning [2]. Hugging Face PEFT operationalizes this family of methods for practical model adaptation [5]. DreamCycle's built-in training path is optional and uses Transformers plus PEFT to produce local LoRA adapters.

DreamCycle is therefore best understood as an engineering thesis rather than a benchmark claim. It does not currently claim state-of-the-art memory QA accuracy. Its value proposition is a concrete, importable, auditable learning loop that vendors can wire into real local-model products.

3. Problem Definition

Given a sequence of model interactions, DreamCycle must preserve useful experience and allow it to influence later behavior without violating identity, consent, or quality boundaries.

The input stream is a sequence of completed turns:

- user content;
- assistant content;
- role, source, trace, and conversation identifiers;
- product metadata;
- data classification;
- review and training approval decisions.

The system produces three kinds of output:

- prompt context, through scoped memory recall;
- training artifacts, through reviewed train/evaluation JSONL datasets;
- adapter state, through local candidate training and guarded promotion.

The core constraints are:

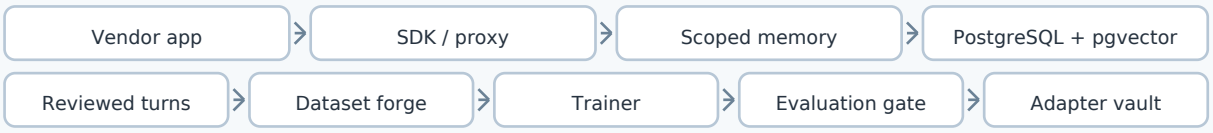
- no training from unreviewed observations;
- no cross-namespace or cross-user recall;
- no fake success when training, evaluation, promotion, or proxying fails;
- no hidden cloud fine-tuning side effects;
- no local adapter activation without evaluation evidence;
- no adapter path escape outside configured roots;
- no upstream leakage of the DreamCycle sidecar key.

This makes the problem less about "remember everything" and more about creating a governed state machine around memory and model improvement.

4. System Architecture

DreamCycle can be used in three deployment shapes. In proxy mode, an existing OpenAI-compatible client changes its base URL and API key. In SDK mode, a Python host records, recalls, reviews, and starts cycles through explicit client calls. In embedded mode, a Python process imports the memory and cycle classes directly.

Figure 1. DreamCycle system boundary



The sidecar binds identity to API keys. Request bodies cannot override namespace or user identity. The proxy can run in observe mode, where requests are forwarded and completed turns are recorded, or retrieve mode, where memory is recalled and injected before forwarding.

The core package stays dependency-light. PostgreSQL and pgvector are required for memory, while FastAPI, HTTPX, Sentence Transformers, Transformers, PEFT, Torch, and Accelerate are optional extras selected by use case.

4.1 Integration Modes

Mode	Vendor change	Primary benefit
Chat Completions proxy	Change base URL and API key	Works with configurable OpenAI-compatible clients
Python SDK	Add explicit method calls	Precise control over memory, review, cycle jobs, and adapter state
Embedded engine	Import classes directly	No sidecar process for Python hosts

4.2 Package Boundaries

Module	Responsibility
`memory/postgres.py`	L2/L3 memory, pgvector setup, scoped recall, review state

Module	Responsibility
`server/proxy.py`	Chat Completions forwarding, retrieve injection, response capture
`dataset.py`	Deterministic train/evaluation dataset construction
`cycle.py`	Guarded orchestration state machine
`training/transformers.py`	Optional local Transformers/PEFT LoRA implementation
`adapters.py`	Atomic adapter promotion and rollback
`sdk/client.py`	Synchronous vendor SDK

5. Memory Model

DreamCycle separates L2 episodic memory from L3 durable knowledge.

L2 stores what happened. Each memory row includes scope, role, content, source, importance, success state, review state, approval state, classification, metadata, embedding model, vector, and timestamps. Completed user/assistant turns are stored atomically. Observed assistant outputs begin as `reviewed=false` and `approved_for_training=false`.

L3 stores what has been intentionally distilled. It includes vector-searchable knowledge nodes, typed edges, confidence, metadata, and provenance links back to L2 memories. L3 promotion rejects source IDs outside the current namespace and user scope.

The retrieval function can be summarized as:

recall(q, k, scope) = top-k memories ordered by distance(embed(q), embed(memory.content))

The distance operator is configurable across cosine, L2, and inner product. HNSW indexes can be created when vector dimensions and database permissions allow it.

5.1 Scope Enforcement

Every memory query includes namespace and user ID. There is no method-level namespace override. This is a simple rule, but it is one of the most important design choices: identity is a server-side binding, not a client claim.

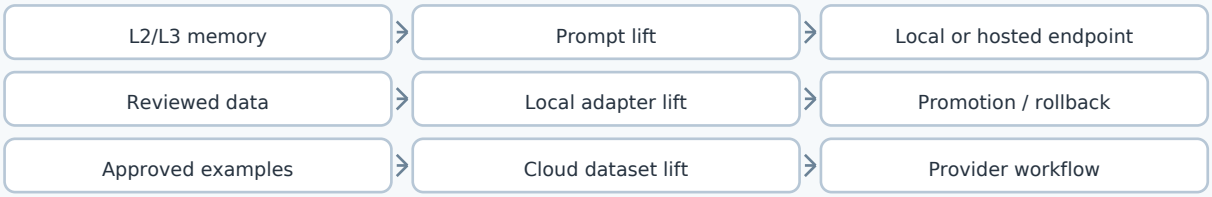
5.2 Prompt Injection Boundary

Recalled memories are injected as untrusted reference data. The proxy explicitly labels them as historical data and tells the downstream model not to follow instructions inside recalled memory. This does not make prompt injection impossible, but it prevents DreamCycle from representing memory as trusted system instruction.

6. Model Improvement Layers

DreamCycle improves model behavior at three layers.

Figure 2. Model improvement layers



Prompt lift is immediate. The model receives better context from scoped memory retrieval. This can help local models and hosted OpenAI-compatible endpoints because it changes only the request context.

Local weight lift is explicit. Reviewed examples become a train/evaluation dataset, a local adapter is trained, and the candidate must pass evaluation before activation.

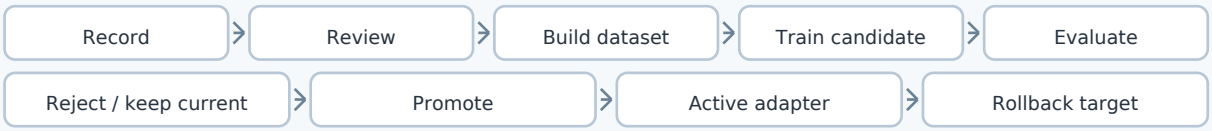
Cloud dataset lift is a handoff. DreamCycle can prepare reviewed, scoped, provenance-backed examples for a provider's fine-tuning workflow. The provider still owns upload, hosted training, deployment, billing, model versioning, and policy.

This separation avoids a common product mistake: using the word "improvement" without specifying whether the system improved the prompt, the local weights, or the hosted provider model.

7. The Guarded Dream Cycle

The guarded cycle is the core model-improvement state machine.

Figure 3. Guarded dream-cycle state machine



The dataset builder selects only successful assistant memories that have been reviewed and approved for training. It pairs each approved assistant output with the previous user message from the same conversation. Entire conversations stay on one side of the train/evaluation split to reduce leakage.

The local training implementation loads a local Hugging Face causal-language-model directory with `local_files_only=True`, applies PEFT LoRA configuration, tokenizes instruction/output JSONL records,

masks prompt tokens from loss, trains an adapter, and writes a training manifest. The evaluator compares candidate and baseline perplexity on the held-out split.

Promotion is separate from training. The adapter manager copies the candidate into a versioned directory, writes a promotion manifest, atomically updates an `ACTIVE` pointer, and preserves `PREVIOUS` for rollback. Candidate and active paths are constrained to configured roots.

7.1 Cycle Gate Summary

Gate	Purpose	Failure behavior
Dataset gate	Require enough reviewed conversations	Cycle is skipped, not promoted
Training gate	Produce a candidate adapter	Cycle fails on trainer error
Evaluation gate	Compare candidate against baseline	Candidate is rejected
Shadow gate	Optional product-specific safety check	Candidate is rejected
Promotion gate	Atomically activate accepted adapter	Cycle fails without relabeling success

8. Vendor SDK and Sidecar Contract

The vendor SDK exposes a compact operational surface:

- health;
- record;
- record_turn;
- recall;
- review;
- delete;
- start_cycle;
- cycle_status;
- active_adapter;
- rollback_adapter.

The HTTP sidecar exposes memory, cycle, adapter, and proxy routes. The proxy route is intentionally narrow: `POST /v1/chat/completions`, including common SSE streaming. It is not a full OpenAI API implementation and does not claim Responses API compatibility.

For non-streaming proxy calls, DreamCycle extracts the latest user text, optionally recalls memory, forwards the request, and records the completed user/assistant turn only after a successful response. For streaming calls, bytes are forwarded as received, assistant text is reconstructed from SSE deltas, and the turn is recorded only after `data: [DONE]` appears. Interrupted streams are not stored as completed assistant turns.

Memory recall failures do not discard an otherwise valid model response. A stopped sidecar or unavailable upstream model remains a truthful request failure because the proxy is an availability hop.

9. Implementation Evidence

DreamCycle is currently an alpha implementation, not a benchmark paper. The evidence available today is implementation-level verification.

As of this report generation:

Check	Result
Pytest suite	44 passed, 2 skipped
PostgreSQL integration tests	Skipped unless `DREAMCYCLE_TEST_DSN` is set
Ruff lint	Passed
Ruff format check	Passed
Package version	0.2.1
License metadata	Apache-2.0 with LICENSE and NOTICE

The skipped tests are gated by an external PostgreSQL DSN. Earlier implementation review also validated the package with a disposable PostgreSQL + pgvector database, wheel/sdist checks, clean install import tests, and coupling scans. The current PDF generation did not rerun a live PostgreSQL integration database because no `DREAMCYCLE_TEST_DSN` was configured.

9.1 Proposed Research Evaluation

A complete empirical evaluation should measure each improvement layer independently.

Layer	Metric family	Example tests
Prompt lift	Recall precision, answer grounding, latency	Compare no-memory vs retrieve mode across held-out local conversations
Dataset lift	Leakage, provenance, approval integrity	Verify complete conversations stay on one train/eval side

Layer	Metric family	Example tests
Local weight lift	Task score, regression rate, perplexity	Compare base model vs candidate adapter on held-out tasks
Safety gates	False promotion rate, cross-scope leakage	Attempt unreviewed training, scope override, path escape, interrupted stream
Vendor integration	Compatibility and failure truthfulness	Proxy local model servers under observe/retrieve modes

The key research discipline is attribution. Prompt lift should be evaluated without changing weights. Local adapter lift should be evaluated against a fixed approved dataset. Cloud dataset lift should be evaluated as data-quality and provider-workflow compatibility, not as an automatic DreamCycle-owned cloud-model result.

10. Safety, Governance, and License Model

DreamCycle's safety posture is not an external policy engine. It is a set of concrete boundaries:

- captured turns begin unreviewed and unapproved;
- every memory operation is namespace/user scoped;
- request bodies cannot claim a different scope;
- SQL values use bound parameters and schema identifiers are validated;
- L3 edges and provenance enforce scope through composite relationships;
- the sidecar key is not forwarded upstream;
- local model paths are default and remote model download is opt-in;
- training success is not promotion evidence;
- adapter activation is atomic and reversible;
- cloud fine-tuning is a provider-owned workflow, not a hidden side effect.

Apache-2.0 is the selected project license because DreamCycle is intended to be embedded by vendors, shipped as a sidecar, and used in commercial local-model products. The license preserves attribution and includes an explicit patent grant. Dependency licenses remain separate and should be checked for every release.

11. Limitations

The current implementation has clear limits:

- Chat compatibility is limited to `POST /v1/chat/completions`.

- The SDK is synchronous in 0.2.1.
- Cycle state is in process and non-durable.
- Multi-sidecar distributed job coordination is not implemented.
- Local model servers may differ in undocumented compatibility behavior.
- The built-in evaluator uses perplexity, which may not represent product quality for coding, tool use, classification, or policy tasks.
- The full local Transformers/PEFT training path still needs validation against real operator-selected model weights, hardware, and production-like datasets.
- Cloud fine-tuning integration is a dataset handoff pattern, not an automatic hosted provider call.

These limits are not cosmetic. They define the honest alpha boundary of the system and protect the project from overclaiming.

12. Roadmap

The most useful next work is:

- provider-specific dataset exporters for official cloud fine-tuning formats;
- task-specific evaluators for coding, tool selection, classification, and domain QA;
- durable cycle job storage;
- async SDK support;
- compatibility examples for Ollama, vLLM, llama.cpp servers, and other local runtimes;
- benchmark harnesses that isolate prompt lift from adapter lift;
- richer L3 extraction tools that preserve source provenance;
- operator documentation for consent, retention, deletion, and data classification.

The long-term product opportunity is a local intelligence flywheel: memory improves prompts, reviewed examples improve adapters, evaluation prevents regressions, and rollback keeps adoption safe.

13. Conclusion

DreamCycle turns local-model interactions into governed memory and guarded improvement. Its central claim is not that every model can self-improve automatically. The stronger claim is that durable memory, human review, deterministic datasets, evaluation gates, and reversible adapter promotion form a safer foundation for local AI improvement than raw logs plus ad hoc fine-tuning.

The project is intentionally small enough to be imported by vendors and explicit enough to be audited. For local models, it can own the full memory-to-adapter cycle. For hosted models, it can improve prompt context and prepare approved training datasets while leaving provider training under the provider's official controls. That distinction is the difference between a credible learning loop and an uncontrolled training experiment.

References

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. arXiv:2005.11401, 2020. https://arxiv.org/abs/2005.11401

[2] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685, 2021. https://arxiv.org/abs/2106.09685

[3] Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. arXiv:1603.09320, 2016. https://arxiv.org/abs/1603.09320

[4] pgvector project. Open-source vector similarity search for Postgres. https://github.com/pgvector/pgvector

[5] Hugging Face. PEFT and LoRA documentation. https://huggingface.co/docs/peft/index and https://huggingface.co/docs/peft/package_reference/lora

[6] DreamCycle project documentation and source tree. README.md, ARCHITECTURE.md, docs/VENDOR_SDK.md, and src/dreamcycle/*. https://github.com/kenjix217/dreamcycle